



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 4: Thuật toán tham lam

PHẦN 1



Thuật toán tham lam là gì?

Thuật toán tham lam là chiến lược giải quyết bài toán bằng cách chọn lựa tốt nhất tại thời điểm hiện tại, không quan tâm đến hậu quả sau này.

Đặc điểm chính:

- Đưa ra quyết định cục bộ tối ưu (local optimal choice)
- Không quay lui, không xem xét lại quyết định
- Hy vọng dẫn đến giải pháp toàn cục tối ưu (global optimal)
- Đơn giản và nhanh, nhưng không phải lúc nào cũng đúng!

Tình huống: Bạn đang ở điểm A, muốn đến điểm B bằng cách đi qua các điểm trung gian.

Chiến lược tham lam:

Tại mỗi bước, chọn điểm gần đích nhất có thể đi đến.

Kết quả:

-  Có thể đến đích nhanh nhất (may mắn!)
-  Có thể đi vào ngõ cụt (không may!)

→ Greedy không đảm bảo lời giải tối ưu cho mọi bài toán.

Greedy-choice property là gì?

Một bài toán có greedy-choice property nếu ta có thể xây dựng lời giải tối ưu toàn cục bằng cách thực hiện các lựa chọn tham lam cục bộ.

Nói cách khác:

Lựa chọn “tốt nhất ngay bây giờ” sẽ không làm hỏng lời giải tối ưu sau này.

Hai yếu tố bắt buộc:

- Greedy choice: Chọn tốt nhất tại thời điểm hiện tại
- Optimal substructure: Bài toán con sau khi chọn vẫn là bài toán tối ưu

Ví dụ bài toán KHÔNG có greedy-choice property

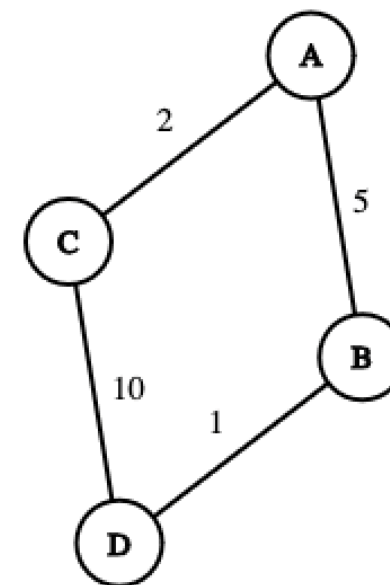
Bài toán: Tìm đường đi ngắn nhất từ A đến D trong đồ thị

Greedy (chọn cạnh nhỏ nhất):

□ $A \rightarrow C (2) \rightarrow D (10) = 12$ ✗

Lời giải tối ưu:

□ $A \rightarrow B (5) \rightarrow D (1) = 6$ ✓



Kết luận: Chọn “tốt nhất ngay bây giờ” (cạnh 2) làm hỏng lời giải tối ưu!

Bài toán Activity Selection

Đề bài: Cho n hoạt động, mỗi hoạt động i có:

- Thời gian bắt đầu: $\text{start}[i]$
- Thời gian kết thúc: $\text{finish}[i]$

Mục tiêu: Chọn số lượng tối đa các hoạt động sao cho không có 2 hoạt động nào chồng lấp về thời gian.

Hai hoạt động i và j không chồng lấp nếu: $\text{finish}[i] \leq \text{start}[j]$ HOẶC $\text{finish}[j] \leq \text{start}[i]$

Ví dụ: Cho các hoạt động

Trực quan hóa timeline:

A1: [1====3]

A2: [2=====5]

A3: [4=====7]

A4: [6=====9]

A5: [5=====8]

Hoạt động	Bắt đầu	Kết thúc
A1	1	3
A2	2	5
A3	4	7
A4	6	9
A5	5	8

Câu hỏi: Chọn được tối đa bao nhiêu hoạt động?

Các chiến lược tham lam có thể?

Ý tưởng 1: Chọn hoạt động bắt đầu sớm nhất?

✗ Sai! Hoạt động bắt đầu sớm có thể kéo dài lâu, chặn nhiều hoạt động sau.

Ý tưởng 2: Chọn hoạt động ngắn nhất (finish - start)?

✗ Sai! Hoạt động ngắn có thể ở giữa, chặn cả 2 bên.

Ý tưởng 3: Chọn hoạt động xung đột ít nhất?

✗ Phức tạp! Phải tính toán xung đột với tất cả hoạt động khác.

Ý tưởng 4: Chọn hoạt động kết thúc sớm nhất?

✓ ĐÚNG! Đây là chiến lược tối ưu!

Tại sao chọn kết thúc sớm nhất?

Chọn hoạt động kết thúc sớm $\rightarrow$ Giải phóng thời gian sớm $\rightarrow$ Còn nhiều thời gian cho các hoạt động sau!

Ví dụ:

Hoạt động A1[1,3]

Hoạt động A2[2,5]

$\rightarrow$ Kết thúc sớm = Linh hoạt hơn!

Thuật toán Greedy cho Activity Selection

- Bước 1: Sắp xếp các hoạt động theo thời gian kết thúc tăng dần (finish time)
- Bước 2: Chọn hoạt động kết thúc sớm nhất (sau khi đã sort)
- Bước 3: Lặp
 - Nếu hoạt động i không xung đột với hoạt động vừa chọn $\rightarrow$ Chọn i
 - Cập nhật thời gian kết thúc của hoạt động cuối cùng đã chọn
- Bước 4: Kết quả

Trả về danh sách các hoạt động đã chọn

Cài đặt bằng Python

```
def activity_selection(activities):  
    """  
    activities: list of tuples (start, finish)  
    Return: list of selected activities  
    """  
  
    # Bước 1: Sort theo finish time  
    activities.sort(key=lambda x: x[1])  
  
    # Bước 2: Chọn hoạt động đầu tiên  
    selected = [activities[0]]  
    last_finish = activities[0][1]  
  
    # Bước 3: Duyệt các hoạt động còn lại  
    for i in range(1, len(activities)):  
        start, finish = activities[i]  
  
        # Kiểm tra không xung đột  
        if start >= last_finish:  
            selected.append((start, finish))  
            last_finish = finish  
  
    return selected
```

Test code

```
# Test case
activities = [
    (1, 3),    # A1
    (2, 5),    # A2
    (4, 7),    # A3
    (6, 9),    # A4
    (5, 8),    # A5
]

result = activity_selection(activities)
print(f"Số hoạt động được chọn: {len(result)}")
print(f"Các hoạt động: {result}")
```

```
Số hoạt động được chọn: 2
Các hoạt động: [(1, 3), (4, 7)]
```

Độ phức tạp thời gian

- Bước 1: Sắp xếp n hoạt động: $O(n \log n)$
- Bước 2: Chọn hoạt động đầu: $O(1)$
- Bước 3: Loop
 - Duyệt n-1 hoạt động: $O(n)$
 - Mỗi lần kiểm tra: $O(1)$
- Tổng độ phức tạp: $O(n \log n) + O(1) + O(n) = O(n \log n)$
- Không gian: $O(1)$ nếu không tính output (sort in-place)

Các biến thể của Activity Selection

Biến thể 1: Mỗi hoạt động có trọng số (giá trị), tối đa hóa tổng giá trị

→ Greedy KHÔNG đúng! Phải dùng Dynamic Programming

Biến thể 2: Có k phòng, mỗi hoạt động cần 1 phòng, tối đa hóa số hoạt động

→ Greedy vẫn đúng! Dùng priority queue

Biến thể 3: Tối thiểu hóa số phòng cần thiết

→ Greedy đúng! Sort theo start time

So sánh với các phương pháp khác

Phương pháp	Độ phức tạp	Kết quả	Ghi chú
Brute Force	$O(2^n)$	Tối ưu	Thử tất cả tập con
Backtracking	$O(2^n)$	Tối ưu	Có pruning, nhanh hơn brute force
Dynamic Programming	$O(n^2)$	Tối ưu	Nếu có trọng số
Greedy	$O(n \log n)$	Tối ưu	Chỉ với unweighted

Kết luận: Greedy nhanh nhất khi áp dụng được!

Khi nào nên dùng Greedy?

Nên dùng Greedy khi:

- Bài toán có greedy-choice property
- Có thể chứng minh tính đúng (exchange argument, induction...)
- Cần tốc độ nhanh ($O(n \log n)$)
- Có thể định nghĩa rõ tiêu chí chọn lựa cục bộ

Lỗi thường gặp khi dùng Greedy

- Lỗi 1: Chọn sai tiêu chí tham lam
 - Chọn bắt đầu sớm nhất → Sai!
 - Chọn ngắn nhất → Sai!
 - Chọn kết thúc sớm nhất → Đúng!
- Lỗi 2: Quên sort trước khi loop.
- Lỗi 3: Không kiểm tra ràng buộc.
- Lỗi 4: Dùng Greedy khi không có greedy-choice property

Lưu ý khi dùng thuật toán tham lam

- Thử nhiều tiêu chí tham lam:
 - Nếu chưa biết tiêu chí nào đúng → thử tất cả!
 - Test với nhiều test case nhỏ
- Vẽ timeline/sơ đồ:
 - Visualization giúp hiểu bài toán rõ hơn
- Chứng minh hoặc test kỹ:
 - Tìm phản ví dụ
 - Nếu không chứng minh được → đừng dùng greedy!

PHẦN 2



Bài toán Coin Change

Cho một số tiền N và tập các mệnh giá tiền xu $\text{coins} = [c_1, c_2, \dots, c_n]$.

Tìm số lượng xu ít nhất để đổi được đúng N đồng.

Input:

- amount: Số tiền cần đổi (N)
- coins: Danh sách các mệnh giá có sẵn

Output:

- Số lượng xu ít nhất
- Hoặc -1 nếu không thể đổi được

Cách tiếp cận Greedy

Chiến lược tham lam: Luôn chọn mệnh giá lớn nhất có thể sử dụng!

Thuật toán:

- Sắp xếp coins giảm dần
- Với mỗi mệnh giá (từ lớn đến nhỏ):
 - Lấy nhiều nhất có thể ($\text{amount} // \text{coin}$)
 - Trừ đi số tiền đã lấy
- Nếu $\text{amount} = 0 \rightarrow$ thành công
- Nếu $\text{amount} > 0 \rightarrow$ không đổi được

Code Greedy cho Coin Change

```
def coin_change_greedy(amount, coins):  
    # Sắp xếp giảm dần  
    coins.sort(reverse=True)  
  
    result = []  
    for coin in coins:  
        while amount >= coin:  
            result.append(coin)  
            amount -= coin  
  
    return result  
  
# Test  
coins = [25,10,5,1]  
amount = 63  
result = coin_change_greedy(amount, coins)  
print(f"Xu cần dùng: {result}")  
print(f"Số lượng: {len(result)}")  
# Output: [25, 25, 10, 1, 1, 1] → 6 xu
```

Greedy cho kết quả đúng

Hệ tiền tệ chuẩn:

Ví dụ 1: Hệ tiền Mỹ

Coins: cent [50,10,5,1]

Amount: 67 cent

Greedy:

$50 \times 1 = 50$, còn 17 $\rightarrow 10 \times 1 = 10$, còn 7 $\rightarrow 5 \times 1 = 5$, còn 2 $\rightarrow 1 \times 2 = 2$, còn 0

Tổng: 5 xu 

Không có cách nào tốt hơn! Greedy = Optimal

Ví dụ 1:

Hệ tiền VN (đơn vị: nghìn đồng):

Coins:[200,100,50,20]

Amount: 370

Greedy:

$200 \times 1 = 200$, còn 170 $\rightarrow 100 \times 1 = 100$, còn 70 $\rightarrow 50 \times 1 = 50$, còn 20 $\rightarrow 20 \times 1 = 20$, còn 0

Tổng: 4 tờ 

Greedy đúng với hệ tiền thật!

Tại sao Greedy đúng với tiền chuẩn?

Mỗi mệnh giá lớn có thể thay thế tối ưu các mệnh giá nhỏ hơn

Ví dụ với: [5,10,25]

- $25 = 2 \times 10 + 5$ (3 xu) $\rightarrow$ dùng 1 xu 25 tốt hơn
- $10 = 2 \times 5$ (2 xu) $\rightarrow$ dùng 1 xu 10 tốt hơn

Thiết kế theo quy tắc:

- Mỗi mệnh giá gấp 2-5 lần mệnh giá nhỏ hơn
- Đảm bảo không có “khoảng trống” làm Greedy sai

Kết luận: Hệ tiền thật được thiết kế để Greedy luôn đúng!

Khi nào Greedy cho kết quả sai?

Giả sử có hệ mệnh giá: $[1, 10, 25]$ (thiếu mệnh giá 5!)

Bài toán: Đổi 30 đồng với ít xu nhất

Greedy chọn:

$25 \rightarrow$ còn 5

$1, 1, 1, 1, 1 \rightarrow$ tổng 6 xu

Lời giải tối ưu:

$10, 10, 10 \rightarrow$ chỉ 3 xu!

$\rightarrow$ Greedy KHÔNG phải lúc nào cũng đúng!

Tại sao Greedy sai

Nguyên nhân: Thiếu mệnh giá 5

- Nếu có $\rightarrow$ Greedy: $25 + 5 = 2$ xu 
- Nhưng với $\rightarrow$ Greedy: $25 + 1 \times 5 = 6$ xu 

Lựa chọn tham lam (25) phá hỏng tương lai:

- Chọn 25 $\rightarrow$ còn 5 $\rightarrow$ chỉ dùng được xu 1
- Không chọn 25 $\rightarrow$ dùng 3 xu 10 $\rightarrow$ tối ưu hơn!

Khi nào dùng Greedy cho Coin Change?

- ✓ Hệ mệnh giá là tiền tệ chuẩn (USD, VND, EUR...)
- ✓ Đã test và verify với nhiều test case
- ✓ Chấp nhận lời giải “đủ tốt” (approximation)
- ✓ Cần tốc độ nhanh $O(n \log n)$

KHÔNG dùng Greedy khi

- ✗ Hệ mệnh giá tùy ý
 - ✗ Cần lời giải tối ưu chính xác 100%
 - ✗ Đề bài là competitive programming (cần AC)
 - ✗ Tìm được test case mà greedy sai
- An toàn nhất: Dùng Dynamic Programming!

Interval Scheduling

Bài toán: Cho n khoảng thời gian (intervals), mỗi khoảng có $[start, end]$.

Các biến thể:

1. Maximum Non-overlapping Intervals

- Chọn tối đa số khoảng không chồng lấp
- Greedy: Sort theo end time 

2. Minimum Intervals to Remove

- Xóa ít nhất bao nhiêu khoảng để còn lại không chồng lấp?
- Greedy: $n - (\text{số khoảng tối đa})$ 

Interval Scheduling

3. Minimum Meeting Rooms

- Tối thiểu bao nhiêu phòng để xếp tất cả intervals?
- Greedy: Sort theo start, dùng heap 

Bài toán: Minimum Meeting Rooms

Đề bài:

- Cho n cuộc họp, mỗi cuộc có $[start, end]$.
- Tìm số phòng họp ít nhất cần thiết để tất cả cuộc họp diễn ra.

Ví dụ:

Meetings: $[[0, 30], [5, 10], [15, 20]]$

Kết quả: Cần 2 phòng như hình bên

```
Room 1: [0=====30]
```

```
Room 2:      [5==10]   [15==20]
```

Chiến lược Greedy

Dùng Min Heap để theo dõi thời gian kết thúc của các phòng đang dùng.

- Sort meetings theo start time
- Heap lưu end time của các phòng đang dùng
- Với mỗi meeting:
 - Nếu meeting bắt đầu sau khi phòng sớm nhất kết thúc → tái sử dụng phòng
 - Ngược lại → cần thêm phòng mới
- Kết quả = số phòng tối đa cần dùng cùng lúc

Code Minimum Meeting Rooms

```
# Test case 1
meetings1 = [[0, 30], [5, 10], [15, 20]]
print(min_meeting_rooms(meetings1)) # Output: 2

# Test case 2
meetings2 = [[1, 5], [8, 9], [8, 9]]
print(min_meeting_rooms(meetings2)) # Output: 2

# Test case 3
meetings3 = [[1, 10], [2, 7], [3, 19], [8, 12], [10, 20], [11, 30]]
print(min_meeting_rooms(meetings3)) # Output: 4

# Giải thích test 3:
# Thời điểm 3-7: 3 meetings cùng lúc [1,10], [2,7], [3,19]
# Thời điểm 8-10: 4 meetings cùng lúc [1,10], [3,19], [8,12], [10,20]
# → Cần 4 phòng
```

```
import heapq

def min_meeting_rooms(meetings):
    """
    Tìm số phòng họp tối thiểu
    """
    if not meetings:
        return 0

    # Sort theo start time
    meetings.sort(key=lambda x: x[0])

    # Heap lưu end time của các phòng đang dùng
    heap = []
    heapq.heappush(heap, meetings[0][1]) # Phòng đầu tiên

    for i in range(1, len(meetings)):
        start, end = meetings[i]

        # Nếu meeting bắt đầu sau phòng sớm nhất kết thúc
        if start >= heap[0]:
            heapq.heappop(heap) # Tái sử dụng phòng

        heapq.heappush(heap, end) # Thêm meeting hiện tại

    return len(heap) # Số phòng = kích thước heap
```

Bài toán ghép cặp (Matching)

Đề bài tổng quát:

Cho 2 tập hợp A và B, mỗi phần tử có một số thuộc tính.

Ghép các phần tử từ A với B sao cho thỏa mãn điều kiện và tối ưu một mục tiêu.

Ví dụ: Bài toán Assign Cookies

- Cho n trẻ em (greed) và m bánh (size)
- Ghép bánh cho trẻ sao cho thỏa mãn tối đa

Dấu hiệu Greedy sẽ sai

Bài toán có trọng số (weights):

- 0/1 Knapsack
- Weighted Activity Selection

Quyết định hiện tại phụ thuộc tương lai:

- Shortest Path với cạnh âm
- Coin Change với mệnh giá lạ

Quy tắc vàng: Luôn chứng minh hoặc test kỹ trước khi dùng Greedy!



Kết thúc